

IS 3063 – Lab 7

Structured Query Language (SQL) — Aggregation, Joins & Subqueries

MyUTSA ID: DXV584

Name: Chase Cantrell

Q1. Write an SQL query to calculate the number of products available for each ProductFinish in the Product_T table. The query should display ProductFinish and the count of products (as ProductCount) for each finish, sorted in ascending order by ProductCount.

(2 pts)

```
SELECT ProductFinish, COUNT(*) AS ProductCount
FROM Product_T
GROUP BY ProductFinish
ORDER BY ProductCount;
```

Q2. Write an SQL query to retrieve each ProductFinish and the average ProductStandardPrice for that finish from the Product_T table. Display only finishes where the average ProductStandardPrice is greater than 400. The result should include ProductFinish and the average price (as AveragePrice).

(2 pts)

```
SELECT ProductFinish, AVG(ProductStandardPrice) AS AveragePrice
FROM Product_T
GROUP BY ProductFinish
HAVING AVG(ProductStandardPrice) > 400;
```

Q3. Write an SQL query to find the CustomerState and the total number of customers in each state from the Customer_T table. Display only states that have more than 2 customers. The result should include CustomerState and the count of customers (as CustomerCount).

(2 pts)

```
SELECT CustomerState, COUNT(*) AS Customercount
FROM Customer_T
GROUP BY CustomerState
HAVING Count(*) > 2;
```

Q4. Write an SQL query to retrieve the OrderID, OrderDate, FulfillmentDate, CustomerName, CustomerCompleteAddress (a concatenated field of CustomerAddress, CustomerCity, CustomerState, and CustomerPostalCode), SalespersonName, and SalespersonTelephone. Join the Order_T, Customer_T, and

Salesperson_T tables, assuming each table has the necessary foreign keys for the joins.

(2 pts)

```
SELECT Order_T.OrderID, Order_T.OrderDate, Order_T.FulfillmentDate,
       Customer_T.CustomerName,
       (Customer_T.CustomerAddress||', '||Customer_T.CustomerCity||', '||
        Customer_T.CustomerState||', '||Customer_T.CustomerPostalCode)
       AS CustomerCompleteAddress,
       SalesPerson_T.SalespersonName,
       Salesperson_T.SalespersonTelephone
FROM
    Order_T JOIN Customer_T ON Order_T.CustomerID = Customer_T.CustomerID
    JOIN Salesperson_T ON Order_T.SalespersonID =
    Salesperson_T.SalesPersonID;
```

Q5. Write an SQL query to retrieve each CustomerID, CustomerName, and the total quantity of products they have ordered. Use a derived table to first calculate the total quantity ordered by each customer, and then join it with the Customer_T table to include the customer name.

(2 pts)

```
SELECT Customer_T.CustomerID, Customer_T.CustomerName,
       TotalOrders.TotalQuantityOrdered
FROM
    Customer_T JOIN (SELECT CustomerID, SUM(OrderedQuantity) AS
    TotalQuantityOrdered
                     FROM OrderLine)T Join Order_T.OrderID =
    Order_T.OrderID
                     GROUP BY CustomerID AS TotalOrders
    ON Customer_T.CustomerID = TotalOrders.CustomerID;
```

Q6. Write an SQL query to retrieve all ProductIDs from the Product_T table along with the corresponding OrderIDs from the OrderLine_T table. Include all products, even if they have not been ordered, using an outer join.

(2 pts)

```
SELECT Product_T.ProductID, Orderline_T.OrderID
FROM Product_T
LEFT OUTER JOIN OrderLine_T ON Product_T.ProductID =
OrderLine_T.ProductID;
```

Q7. Write an SQL query to retrieve pairs of EmployeeIDs and EmployeeNames from the Employee_T table for employees who report to the same manager. Ensure that each pair contains two different employees who share the same EmployeeSupervisor.

(2 pts)

```

SELECT E1.EmployeeID, E1.EmployeeName, E2.EmployeeID, E2.EmployeeName
FROM Employee_T E1
      JOIN Employee_T E2 ON E1.EmployeeSupervisor = E2.EmployeeSupervisor
WHERE E1.EmployeeID > E2.EmployeeID;

```

Q8. Write an SQL query to retrieve the CustomerID and CustomerName of customers who have placed at least one order, using the EXISTS clause to check for matching orders in the Order_T table.

(2 pts)

```

SELECT CustomerID, CustomerName
FROM Customer_T
WHERE EXISTS (
    SELECT 1
    FROM Order_T
    WHERE Customer_T.CustomerID = Order_T.CustomerID
);

```

Q9. Write an SQL query to retrieve each ProductID and ProductDescription from the Product_T table where the product's ProductStandardPrice is greater than the average ProductStandardPrice of all products with the same ProductFinish. Use a correlated subquery to compare each product's price with the average price in its ProductFinish category.

(2 pts)

```

SELECT ProductID, ProductDescription
FROM Product_T P1
WHERE ProductStandardPrice > (
    SELECT AVG(ProductStandardPrice)
    FROM Product_T P2
    WHERE P1.ProductFinish = P2.ProductFinish
);

```

Q10. Write an SQL query to retrieve the CustomerID and CustomerName of the customer who purchased the highest quantity of the product 'Birch Coffee Table' and the customer who purchased the highest quantity of the product 'Cherry End Table', respectively. Use the Order_T, OrderLine_T, Product_T, and Customer_T tables and UNION to find the required information.

(2 pts)

```

SELECT Product_T.ProductDescription, Customer_T.CustomerID,
       Customer_T.customerName, OrderLine_T.OrderedQuantity
FROM Customer_T
      JOIN Order_T ON Customer_T.CustomerID = Order_T.CustomerID
      JOIN Order_Line_T ON Order_T.OrderID = OrderLine_T.OrderID
      JOIN Product_T ON OrderLine_T.ProductID = Product_T.ProductID

```

```

WHERE Product_T.ProductDescription = 'Birch Coffee Tables'
AND OrderLine_T.OrderedQuantity = (
    SELECT MAX(OrderedQuantity)
    FROM OrderLine_T
        JOIN Order_T ON OrderLine_T.OrderID = Order_T.OrderID
        JOIN Product_T ON OrderLine_T.ProductID = Product_T.ProductID
    WHERE Product_T.ProductDescription = 'Birch Coffee Tables'
)
UNION
SELECT Product_T.ProductDescription, Customer_T.CustomerID,
    Customer_T.customerName, OrderLine_T.OrderedQuantity
FROM Customer_T
    JOIN Order_T ON Customer_T.CustomerID = Order_T.CustomerID
    JOIN Order_Line_T ON Order_T.OrderID = OrderLine_T.OrderID
    JOIN Product_T ON OrderLine_T.ProductID = Product_T.ProductID
WHERE Product_T.ProductDescription = 'Cherry End Table'
AND OrderLine_T.OrderedQuantity = (
    SELECT MAX(OrderedQuantity)
    FROM OrderLine_T
        JOIN Order_T ON OrderLine_T.OrderID = Order_T.OrderID
        JOIN Product_T ON OrderLine_T.ProductID = Product_T.ProductID
    WHERE Product_T.ProductDescription = 'Cherry End Table'
);

```

Total Points: 20 pts